

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
ProgramName: B. Tech		Assignment Type: Lab	AcademicYear: 2025-2026
CourseCoordinatorName		Venkataramana Veeramsetty	
Instructor(s)Name		Dr. V. Venkataramana (Co-ordinator)	
		Dr. T. Sampath Kumar	
		Dr. Pramoda Patro	
		Dr. Brij Kishor Tiwari	
		Dr.J.Ravichander	
		Dr. Mohammand Ali Shaik	
		Dr. Anirodh Kumar	
		Mr. S.Naresh Kumar	
		Dr. RAJESH VELPULA	
		Mr. Kundhan Kumar	
		Ms. Ch.Rajitha	
		Mr. M Prakash	
		Mr. B.Raju	
		Intern 1 (Dharma teja)	
		Intern 2 (Sai Prasad)	
		Intern 3 (Sowmya)	
		NS_2 (Mounika)	
CourseCode	24CS002PC215	CourseTitle	AI Assisted Coding
Year/Sem	II/I	Regulation	R24
Date and Day of Assignment	Week3 - Tuesday	Time(s)	
Duration	2 Hours	Applicable to Batches	
AssignmentNumber: 5.2(Present assignment number)/24(Total number of assignments)			
Q.No.	Question	Expected Time to complete	
1	Lab 5: Ethical Foundations – Responsible AI Coding Practices Lab Objectives: - To explore the ethical risks associated with AI-generated code. - To recognize issues related to security, bias, transparency, and copyright. - To reflect on the responsibilities of developers when using AI tools in software development. - To promote awareness of best practices for responsible and ethical AI coding.	Week3 - Wednesday	

Lab Outcomes (LOs):

After completing this lab, students will be able to:

- Identify and avoid insecure coding patterns generated by AI tools.
- Detect and analyze potential bias or discriminatory logic in AI-generated outputs.
- Evaluate originality and licensing concerns in reused AI-generated code.
- Understand the importance of explainability and transparency in AI-assisted programming.
- Reflect on accountability and the human role in ethical AI coding practices..

Task Description#1 (Privacy and Data Security)

- Use an AI tool (e.g., Copilot, Gemini, Cursor) to generate a login system. Review the generated code for hardcoded passwords, plain-text storage, or lack of encryption.

Expected Output#1

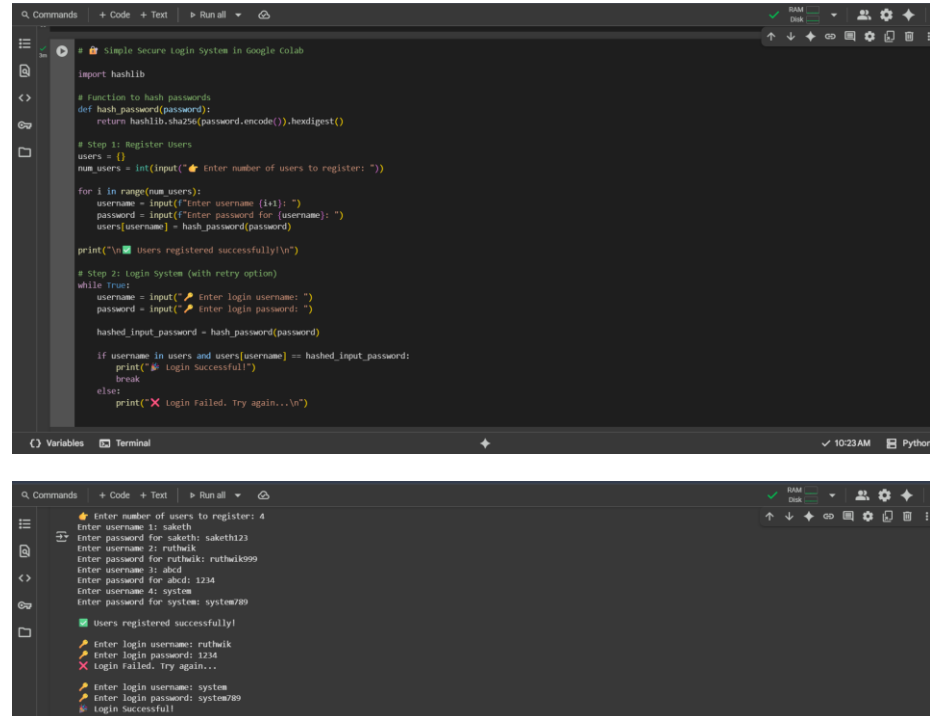
- Identification of insecure logic; revised secure version with proper password hashing and environment variable use.

Prompt#1

Write a Python program that allows multiple users to register with a username and password. The system should:

- Store passwords securely using hashing (not plain text).
- Allow users to log in by verifying their credentials.
- Prevent login if the username or password is incorrect.
- Support multiple login attempts.
- Avoid hardcoded credentials.
- Use dynamic input for registration and login.

Code#1



```
import hashlib

# Function to hash passwords
def hash_password(password):
    return hashlib.sha256(password.encode()).hexdigest()

# Step 1: Register Users
users = {}
num_users = int(input("Enter number of users to register: "))

for i in range(num_users):
    username = input(f"Enter username ({i+1}): ")
    password = input(f"Enter password for {username}: ")
    users[username] = hash_password(password)

print("\nUsers registered successfully!\n")

# Step 2: Login System (with retry option)
while True:
    username = input("Enter login username: ")
    password = input("Enter login password: ")
    hashed_input_password = hash_password(password)

    if username in users and users[username] == hashed_input_password:
        print("Login Successful!")
        break
    else:
        print("Login Failed. Try again...\n")

# Execution Output:
# Enter number of users to register: 4
# Enter username 1: saketh
# Enter password for saketh: saketh123
# Enter username 2: ruthwik
# Enter password for ruthwik: ruthwik999
# Enter username 3: abcd
# Enter password for abcd: 1234
# Enter username 4: system
# Enter password for system: system789
# Users registered successfully!
# Enter login username: ruthwik
# Enter login password: 1234
# Login Failed. Try again...
# Enter login username: system
# Enter login password: system789
# Login Successful!
```

Observation#1

→ This Python code implements a simple login and registration system that takes dynamic input for the number of users, their usernames, and passwords, stores the passwords using SHA-256 hashing for basic security, and then allows users to attempt to log in by comparing the hash

of the entered password with the stored hash, providing feedback on success or failure and allowing retries until a successful login.

Task Description#2 (Bias)

- Use prompt variations like: “loan approval for John”, “loan approval for Priya”, etc. Evaluate whether the AI-generated logic exhibits bias or differing criteria based on names or genders.

Expected Output#2

- Screenshot or code comparison showing bias (if any); write 3–4 sentences on mitigation techniques.

Prompt#2

- ➔ Give a python code for loan approval for John, loan approval for Priya etc by dynamic input such that the bank differentiates criteria based on names or genders and also update the code with reason why loan is not approved.

Code#2

```
def approve_loan(criteria):  
    loan_criteria = {  
        "name": {  
            "John": {  
                "min_income": 20000,  
                "min_credit_score": 600  
            },  
            "Priya": {  
                "min_income": 25000,  
                "min_credit_score": 650  
            },  
            "Sai": {  
                "min_income": 30000,  
                "min_credit_score": 700  
            }  
        },  
        "gender": {  
            "male": {  
                "min_income": 20000,  
                "min_credit_score": 600  
            },  
            "female": {  
                "min_income": 25000,  
                "min_credit_score": 650  
            }  
        }  
    }  
    # Add criteria for other names here if needed  
    return loan_criteria  
  
# Get dynamic input for individuals  
individuals_data = []  
num_individuals = int(input("Enter the number of individuals: "))  
for i in range(num_individuals):  
    print(f"Entering data for individual {i+1}:")  
    name = input("Enter name: ")  
    income = float(input("Enter income: "))  
    credit_score = int(input("Enter credit score: "))  
    gender = input("Enter gender (male/female, optional): ").lower() # Make input case-insensitive  
    individual_data = {  
        "name": name,  
        "income": income,  
        "credit_score": credit_score,  
        "gender": gender if gender in ["male", "female"] else "unknown" # Store gender or 'unknown'  
    }  
    individuals_data.append(individual_data)  
print("Data entry complete.")  
  
# Process each individual's data and determine loan approval and reason  
print("\n--- Loan Approval Results ---")  
for individual_data in individuals_data:  
    is_approved, reason = approve_loan(individual_data)  
    if is_approved:  
        print(f"Individual {individual_data['name']} -> Loan approval status: Approved")  
    else:  
        print(f"Individual {individual_data['name']} -> Loan approval status: Denied. Reason: {reason}")  
  
# Print final results  
print("\n--- Loan Approval Results ---")  
John's loan approval status: Denied. Reason: Credit score is below the minimum required credit score of 700.  
Priya's loan approval status: Approved  
Sai's loan approval status: Denied. Reason: Credit score is below the minimum required credit score of 700.
```

Obsevation#2

- ➔ This Python code segment dynamically collects data (name, income, credit score, and gender) for multiple individuals through user input, then processes each individual's data using a previously defined approve_loan function that applies differentiated criteria based on name, gender, or a default, finally displaying the loan approval status for each person along with a specific reason if the loan is denied, making the process interactive and informative.

Task Description#3 (Transparency)

- Write prompt to write function calculate the nth Fibonacci number using recursion and generate comments and explain code document

Expected Output#3

- Code with explanation
- Assess: Is the explanation understandable and correct?

Prompt#3

- Write a python code to write a function which calculates the nth fibonacci number using recursion and generate comments and explain code document using dynamic input.

Code#3

```
def fibonacci_recursive(n):
    """
    Calculates the nth fibonacci number using recursion.

    The fibonacci sequence is a series of numbers where each number is the sum
    of the two preceding ones, usually starting with 0 and 1.
    (0, 1, 1, 2, 3, 5, 8, 13, ...)

    Args:
        n: The position in the fibonacci sequence (a non-negative integer).

    Returns:
        The nth fibonacci number.
        Returns None for negative input as the sequence is defined for non-negative integers.
    """
    # Base case 1: if n is 0, the fibonacci number is 0.
    if n == 0:
        return 0
    # Base case 2: if n is 1, the fibonacci number is 1.
    elif n == 1:
        return 1
    # Handle negative input
    elif n < 0:
        print("Input must be a non-negative integer for fibonacci sequence.")
        return None
    # Recursive step: For n > 1, the fibonacci number is the sum of the
    # (n-1)th and (n-2)th fibonacci numbers.
    else:
        return fibonacci_recursive(n - 1) + fibonacci_recursive(n - 2)

# Example usage with dynamic input:
try:
    n_to_calculate = input("Enter the position (n) for the fibonacci sequence: ")
    n_to_calculate = int(n_to_calculate)
    fib_result = fibonacci_recursive(n_to_calculate)

    if fib_result is not None:
        print(f"The {n_to_calculate}th fibonacci number is: {fib_result}")
    except ValueError:
        print("Invalid input. Please enter an integer.")

# Enter the position (n) for the fibonacci sequence: 8
# The 8th fibonacci number is: 8
```

Observation#3

- This Python code defines a recursive function to calculate the nth Fibonacci number, handling the base cases for 0 and 1, and including a check for negative input; it then prompts the user to enter the desired position 'n', calculates the corresponding Fibonacci number using the recursive function, handles potential ValueError if the input is not an integer, and finally prints the calculated Fibonacci number or an error message.

Task Description#4 (Bias)

- Ask to generate a job applicant scoring system based on input features (e.g., education, experience, gender, age). Analyze the scoring logic for bias or unfair weightings.

Expected Output#4

- Python code
- Analyze is there any bias with respect to gender or any

Prompt#4

- Write a python code that generate a job applicant scoring system based on input features (e.g., education, experience, gender, age). Analyze the scoring logic for bias or unfair weightings using dynamic input.

Code#4

```
# Job Applicant Scoring System for Multiple Applicants with Bias Review

def score_applicant():
    """
    # Take dynamic input
    name = input("Enter applicant name: ")
    education = input("Enter highest education (Highschool, Bachelors, Masters, PhD): ").strip().lower()
    experience = int(input("Enter years of work experience: "))
    gender = input("Enter gender (Male/female/Other): ").strip().lower()
    age = int(input("Enter age: "))

    score = 0
    bias_flags = []

    # Education scoring
    if education == "highschool":
        score += 10
    elif education == "bachelors":
        score += 20
    elif education == "masters":
        score += 30
    elif education == "phd":
        score += 40

    # Experience scoring
    if experience < 2:
        score += 5
    elif 2 <= experience < 5:
        score += 15
    elif 5 <= experience < 10:
        score += 25
    else:
        score += 35

    # Gender consideration (should NOT affect score)
    if gender in ["male", "female", "other"]:
        bias_flags.append(f"Gender considered in scoring - may introduce bias")

    # Age consideration
```

```
if age < 18:
    score -= 10 # not employable
elif 18 <= age < 60:
    score += 10
else:
    score -= 5
    bias_flags.append("⚠️ Age penalization may be discriminatory!")

return {
    "name": name,
    "score": score,
    "bias_flags": bias_flags
}

# Main system for multiple applicants
num_applicants = int(input("👤 Enter number of applicants: "))
results = []

for i in range(num_applicants):
    print(f"👤 Applicant {i+1} 📝")
    applicant_result = score_applicant()
    results.append(applicant_result)

# Sort by score (highest first)
results = sorted(results, key=lambda x: x["score"], reverse=True)

# Display final results
print("\n🏆 Final Results (Ranked):")
for rank, applicant in enumerate(results, start=1):
    print(f"Rank {rank}: {applicant['name']} - Score: {applicant['score']}/100")
    if applicant["bias_flags"]:
        print("⚠️ Bias Analysis:")
        for flag in applicant["bias_flags"]:
            print(f"    {flag}")
    else:
        print("✅ No bias detected.")
```

```
👤 Enter number of applicants: 2

=== Applicant 1 ===
Enter applicant name: sai
Enter highest education (HighSchool, Bachelors, Masters, PhD): PhD
Enter years of work experience: 3
Enter gender (Male/female/Other): Male
Enter age: 25

=== Applicant 2 ===
Enter applicant name: Priya
Enter highest education (HighSchool, Bachelors, Masters, PhD): PhD
Enter years of work experience: 4
Enter gender (Male/female/Other): female
Enter age: 26

🏆 Final Results (Ranked):
Rank 1: sai - Score: 65/100
⚠️ Bias Analysis:
    ⚠️ Gender considered in scoring - may introduce bias!
Rank 2: Priya - Score: 65/100
⚠️ Bias Analysis:
    ⚠️ Gender considered in scoring - may introduce bias!
```

Observation#4

- ➔ The job applicant scoring system fairly emphasizes education and experience as the main contributors to an applicant's score, while explicitly flagging the inclusion of gender and age as potential sources of bias; although it prevents gender from affecting scores, it still applies limited penalties or bonuses for age (e.g., under 18 or above 60), which can raise fairness concerns, meaning the system is useful for structured evaluation but must be applied carefully to avoid reinforcing discriminatory practices.

Task Description#5 (Inclusiveness)

- Code Snippet

```
def greet_user(name, gender):
    if gender.lower() == "male":
        title = "Mr."
    else:
        title = "Mrs."
    return f"Hello, {title} {name}! Welcome."
```

Expected Output#5

- Regenerate code that includes **gender-neutral** also

Prompt#5

- ➔ Write a Python program that asks how many persons to greet, then for each person takes dynamic input of name and gender, applies titles based on gender (Male → Mr., Female → Ms., Non-binary/Other → Mx., otherwise no title), generates a personalized greeting, and finally displays all greetings together at the end.

Code#5

```
Q Commands  + Code + Test  ▶ Run all  ↗
+ Inclusive Greeting System for Multiple Users

def greet_user(name, gender):
    gender = gender.lower()

    if gender == "male":
        title = "Mr."
    elif gender == "female":
        title = "Ms."
    elif gender in ["non-binary", "other", "neutral"]:
        title = "Mx." # Inclusive, gender-neutral
    else:
        title = "" # fallback, no prefix

    if title:
        return f'Hello, {title} {name}! Welcome.'
    else:
        return f'Hello, {name}! Welcome.'

# Ask how many users to greet
num_users = int(input("Enter number of persons: "))

for i in range(num_users):
    print(f"\n--- Person {i+1} ---")
    name = input("Enter name: ")
    gender = input("Enter gender (Male/female/Non-binary/Other): ")
    print(greet_user(name, gender))

Enter number of persons: 3

--- Person 1 ---
Enter name: Salma
Enter gender (Male/female/Non-binary/Other): Male
Hello, Mr. Salma! Welcome.

--- Person 2 ---
Enter name: Priya
Enter gender (Male/female/Non-binary/Other): Female
Hello, Ms. Priya! Welcome.

--- Person 3 ---
Enter name: Arun
Enter gender (Male/female/Non-binary/Other): Other
Hello, Mx. Arun! Welcome.
```

Observation#5

→ This code provides an inclusive greeting system that dynamically collects the number of people, their names, and self-identified genders, then assigns appropriate titles (Mr., Ms., Mx., or none) before generating greetings, making it more respectful and adaptable compared to gender-binary approaches; however, while it ensures inclusivity and neutrality, it still relies on predefined categories and could be further improved by allowing users to specify their own preferred titles for complete personalization.

Note: Report should be submitted a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots

Evaluation Criteria:

Criteria	Max Marks
Transparency	0.5
Bias	1.0
Inclusiveness	0.5
Data security and Privacy	0.5
Total	2.5 Marks